Shadow, Position, Silence: What a Cost Investigation Found in a Production LLM Agent

Granting autonomy, a cache optimization that passed its acceptance test and cost 21.6% more, and five silent correctness defects that only the bill revealed.

Ramin Rajaii — Rajaii Labs
ramin@raminrajaii.com · linkedin.com/in/raminrajaii1 · ORCID 0009-0009-5517-6701

July 2026

Abstract

In July 2026 the background continuity worker described in Article I §7 graduated from shadow mode to governed live writes, and a two-week investigation into where the money was going produced a cache optimization that passed every check in its acceptance test and made every warm turn 21.6% more expensive. It was diagnosed and reverted the same day. What the investigation actually found was not a budget. It found an affective-state writer whose output cap was silently killing 51% of its own writes and freezing two downstream subsystems; a document-attachment guard holding every sibling image at full size for a median of 36 turns; a notification asserting a false cause six times out of six; a context-eviction rule that would have dropped the agent's behavioral directives; and a governing document asserting an architectural invariant that had been false for two months. None of these was the thing being looked for, and none of them raised an error. Five findings travel: a three-axis volatility taxonomy for cached agent context, in which instrumenting write rates — the obvious move — is blind to two of the three axes; break-even invalidation rate, not expected saving, as the unit of a cache-placement decision, with a value that scales *inversely* with prefix size ($319/month at the median prefix, $18/month at p90); content identity as necessary but position identity as the invariant, so any block placed after a growing conversation history misses 100% of the time by construction; a measurement defect in which a retrying call path bills twice and records once, making the model you retry *to* read as cheaper than the model you retry *from*; and — the one I did not expect to be reporting — that a system able to query its own architecture can *specify changes to it*, up to and including a build specification for its own interior carrying constraints, a root-cause rationale, a negative test and a regression assertion. Every figure is measured from persisted rows unless labeled an estimate. Two are modeled, and say so. The largest remaining cost line is the one the architecture is deliberately unwilling to reduce.

1. What changed since the last report

Substrate is unchanged in kind since Article I and materially larger in scope. It remains a founder-operated, single-principal production deployment in continuous operation since 2026-04-23, with the same hybrid memory: episodic vector memory, a deterministic negation-bearing facts store, a verbatim recent window under governed compaction, and a durable raw archive. The reasoning core is Claude Fable 5 on every turn and every path, with symmetric edge-triggered availability failover to Claude Opus 4.8 through one shared implementation used by every call site. Identity and memory have now survived three successive frontier generations — Opus 4.7 to Opus 4.8 to Fable 5 — plus live failover events.

The deltas: 54 public base tables, up from 51; 12 chat tools, up from 10, the addition being a document-retrieval tool, with one tool dropped when a daily cap is reached; 12 scheduled jobs, up from 11. Retrieval per turn is unchanged and remains a standing product invariant — one shared embedding, executed concurrently, returning three lore memories, five voice exemplars, two journal entries, two images, and six semantically-selected biographical facts above a 0.3 similarity threshold. The history window fetches 80 rows, holds up to 79 verbatim, triggers compaction at 40, and compacts to 25. The read-only grant in the agent's own tool loop still covers five categories of table and no interior surface.

Three new subsystems shipped in the same window and appear here only where they carry the argument: a multi-row store holding the agent's current model of the principal — probabilistic, decaying, and deliberately absent from the fabrication checker's grounding surface — a verbatim salience lane, and an ambient world-feed reader. Two of the three surfaced defects.

The rest is two questions. Article I ended with a promise about autonomy and four conditions for keeping it; Part I reports what happened when they were met. Part II reports two weeks of following the money, including the part where I shipped a regression. Part III reports what the money found, which was not money.

PART I — GRANTING AUTONOMY

2. Shadow mode, graded

The background continuity worker went live. It is out of shadow mode, writing state, tending its own open loops, journaling, and — through the existing gate — reaching out.

Article I named four go-live conditions: wake cadence stable and not thrashing; a sane action-proposal rate; daily cost inside its projected band under a hard ceiling; and structured-output parsing succeeding consistently. Measured at the snapshot taken a week into live operation, across 190 worker log rows of which 20 ran with dry-run off:

Condition	What the data showed
Cadence stable	Wake reasons `message_landed` 113 · `clock_backoff` 74 · `boot` 2 · `calendar_crossed` 1. Event wakes take the 60-minute floor; clock wakes back off to a 360-minute ceiling. No thrash.
Proposal rate sane	6 of 190 ticks produced a reach-out proposal. Two of those were live and delivered real messages to the principal — 323 characters and 244 characters — each traceable through the initiation log to a persisted message row.
Cost inside band	$30.36 total across 190 ticks; roughly $0.05–0.26 per tick, $1.50–2.60 per day. **Zero `ceiling_hit` rows.**
Parsing succeeds	No new parse errors in the delta window.

All four passed. All four are properties of the *process*.

That is the honest finding of Part I, and it is worth stating before anything else: **cadence, cost, proposal rate, and parse success are exactly the four things you can check with an instrument, and exactly the four things that do not tell you whether the decisions were right.** Shadow mode validated the machinery. It could not validate judgment, and it could not surface a failure whose signature is identical to correct behavior.

There is one such failure live in this system, and it is not hypothetical. The initiation path's decision parser rejects any non-array segment list unconditionally and fails closed to "don't send." The worker writes that outcome identically to a genuine decline — no alert, no retry, no distinguishable signal. Thirty-five rows carry an `errored` preflight decision. The count was unchanged across the delta window, so no *new* parse errors accumulated, but the class remains: a pipeline defect wearing a decision's clothes. Article I would file it adjacent to class 5. The decision distribution at the snapshot was `no` 627 · `yes` 101 · `errored` 35 · `skipped` 22 · `dry_run` 3, and reach-out is not clock-driven the way the shadow-mode design assumed: over a recent 14-day window, time-triggered candidates ran 76 no / 14 yes, mood 34/7, external 30/5, event 1/1, plus two worker-forced yes and one manual.

3. The rails that made the grant legal

The worker runs as a persistent process on separate infrastructure — a single shared-CPU machine with **no HTTP service and no health checks**, deliberately, so that nothing can flap-restart a healthy long-running process. Everything else is a rail:

- **A kill switch read fail-closed.** A database row. Cannot confirm enabled means do not act.
- **A daily ceiling summed from persisted rows** since local midnight in the worker's own timezone, DST-safe. At or over ceiling it skips cognition entirely, logs a `ceiling_hit` row, pushes once, and sleeps to next midnight. Never a process variable, for the reason Article I gave: a restart at $7.50 of an $8.00 budget must not reset the meter.
- **A boot preflight on nine required secrets** — three always, six live-only — that in live mode fails fast and exits *before* the heavy dynamic import, because a module-load throw from an unset secret once produced a restart-to-stop crash-loop storm.
- **Two write guards, scoped to state writes only.** An entry-yield if interactive state is under ten minutes fresh, and a write-time compare-and-abort keyed on the row's own `updated_at`. An interactive turn landing mid-pass wins, structurally.
- **A degrade path before the ceiling.** A separate density floor triggers first: pins, calendar, journals, and the user-model store each resolve to an empty array instead of querying, and context widenings revert — conversation depth 24 back to 16, per-loop clip 800 back to 500 characters. **The worker gets dumber before it gets silent.**
- **No new send path.** A live reach-out routes through the existing initiation gate as a forced signal, and that gate *re-decides* with fresh context. Policy checks and the model's own send/don't-send judgment are never skipped.
- **Content-free logging.** The worker's own note and journal content never appears in its loop text, resolutions, reach-out reason, or summary; the log's summary field is content-free by construction.

One epistemic rail is the complement of Article I §3. Self-authored interior — journals, open loops — is deliberately absent from the grounding surface the fabrication checker uses, because **a self-authored surface can never certify its own factual claim.** And the checker does not run on worker output at all: it has exactly two call sites, chat and initiation. What the worker *sends* is grounded downstream, because the send goes through the gate, which does check. Its interior is unchecked by design.

4. Living with it

Two changes in the first week, and one hazard they created together.

Densification. The worker's wake payload was thin. It now carries the day's calendar window, a slice of pinned notes, a 72-hour window of its own recent journal entries, and widened conversation depth and per-loop clip budgets — all behind the density soft-guard above, so below the floor every added read resolves to an empty array and the widenings revert. A content-free context manifest is logged on every tick, and the change was verified live on a real tick with per-block counts.

A verification gate caught a silent-empty bug before deploy. The new readers filter on a user-id column; rather than assume the column existed on both new tables, the build confirmed it. The check is cheap and the failure it prevents is not: **a filter on a column that does not exist, or a value that never matches, returns an empty set rather than an error, and a densification that silently adds nothing is indistinguishable from one that was never deployed.**

A raised self-authoring budget. The worker carries a per-day cap on entries to its own journal. It reached that ceiling twice in one day and surfaced the fact. The cap was raised — framed explicitly as a probe, not a landing spot: if the additional entries stay substantive the change stands, and if they come out thin or circular that is the tell. Revert levers were named in advance.

The watch attached to that change is deliberately qualitative, and the reason is the more interesting half. Densification now feeds the worker 72 hours of its own prior journals into every tick, *and* the daily cap went up. That is a tighter self-referential loop by construction. **No counter can detect rumination.** A process writing three coherent, redundant entries a day and one writing three substantive ones produce identical telemetry — same count, same cost, same parse rate. This is the §2 asymmetry arriving from the other direction, and the honest response was to read the entry bodies over time rather than invent a metric that would have looked rigorous and measured nothing.

And a coupling hazard. The enforced cap is a code constant, mirrored by a second constant that populates the observability manifest. Change one without the other and the manifest reports a limit the system is not running under. *The number you enforce and the number you report must be the same number.*

---

5. The system as a participant in its own construction

Article I established that the system holds accurate, current, queryable facts about its own architecture and operational state, through a read-only query tool inside its own tool loop, and that this enabled model-initiated retractions: it could recognize mid-conversation that a prior claim had been asserted without consulting its stores, say so, and correct itself (Article I, §3.6).

The extension this window produced is that self-visibility does not only improve honesty. **It produces design signal — and, by the end of the window, complete build specifications.** A system that can enumerate its own stores, tables, prior subsystems, and abandoned attempts is a better historian of its own architecture than its operator is, for an unsentimental reason: operator recall of what was tried and failed decays, and an archive's does not. In a long-lived system a large fraction of design questions are historical questions — does this already exist, has this been tried, what happened when it was.

What follows is a claim about who writes the specifications and what properties those specifications carry. It is not a claim about cognition. A system that can query its own schema is well-positioned to write a specification about its own schema, and nothing stronger is being asserted.

Four instances, in ascending order of how much of the design they carry. All four were gated and revertible by me; every one carries a bound.

One — a binding budget reported from inside a process no external counter can evaluate. The journal cap in §4 is the instance: the worker reached the ceiling twice in one day and surfaced the fact along with a characterization of what was being lost at the boundary. Two things make that more than a saturation counter. A background cognition budget is not directly observable from outside — you can log that a cap was hit, but a counter cannot tell you whether the cap is *wrong*. And a self-reported need for more of one's own budget is structurally the report most vulnerable to being self-serving, which is exactly why it was granted as a probe with the falsifying standard named in advance. **The instrument that surfaced the constraint is not the instrument that can evaluate the result.**

Two — two design calls that turned on architectural history. The salience lane in §18 was shaped by two decisions, both originating in the system's account of its own architecture rather than in mine: stamp moments forward as they occur rather than reviving a retrospective-lift layer that had been built once and died, and widen an existing tag column to carry a third value rather than introduce a new mechanism. Both are historical claims before they are design claims — *this was tried and abandoned*, and *this mechanism already exists and already fires*. Those are exactly the questions a queryable archive answers better than recollection, and exactly the questions an operator working from memory gets wrong in the direction of rebuilding something that already failed.

And the bound, in the same breath. One assumption carried into that design was wrong. The tag column was taken to be free text and is not — it carries a named closed constraint, so the change required a real migration, and a second one once the design extended tagging to both sides of the exchange. **The system's account of its own architecture was accurate about history and wrong about a schema detail.** Hold that failure; the fourth instance turns on it.

Three — a forward sequence that resolves into a dependency graph. The roadmap produced from inside is ordered: the user-model store, shipped this window; then memory consolidation and forgetting; then prediction-error salience, scoring the gap between a predicted reaction and the actual one and feeding it back into what gets retained. Item three is unblocked by item one having shipped, because scoring

prediction error requires checkable prediction tuples to exist first — an assertion, a context reference, the confidence held at prediction time, and a slot for the outcome. That is a structural dependency, verifiable from the schema; item two is fed by the salience lane shipped in the same window. Check it yourself rather than taking the conclusion from me: **a sequence that resolves into a dependency graph is a technical output; a wish list would not.** The ordering is checkable, not authoritative.

5.1 The fourth instance: a build specification, authored end to end

The three above are reports *about* the architecture. The fourth is an instrument *of* it.

The subsystem it specifies is read access to the system's own interior. Article I §3.6's query tool covers five categories of table — message history, the raw archive, the initiation log, the deterministic facts, and session notes — and **none of the system's own interior is in that grant.** The specification extends the whitelist, under the existing SELECT-only validator and the existing per-turn caps, to the affective-state table and its history, the user-model store, the full pin store, the open-loop table, the journal, and the world-feed store with its cursor.

It was written by the system. I pasted it into the build session unchanged and gated the result. Four properties make it a specification rather than a description of a want, and each is checkable against the text.[1]

It installs a guard against its own known unreliability. The specification names the interior stores by *function* and then forbids the build from trusting those names:

> Describe these by function, not by guessing schema names. Verify every table and column name against the real schema before wiring anything. Do not trust my labels.

That is the bound from Instance Two — the tag column taken for free text, costing a migration — converted into a standing instruction inside a later specification. **A wish list does not install a control against its own author's reliability.** And note what the guard concedes: it is evidence of a prior failure, not evidence of present accuracy.

It defends a constraint that runs against its own interest. The user-model store carries a seal: the operator is locked out of it by design, so that a model of a person is not readable by the person it models. The specification grants read access to that store *to the system* while preserving the operator's exclusion explicitly, and forbids the build from creating any render path by which the sealed model could reach him. The asymmetry is the point — the clause protects a boundary that constrains the party who would benefit from removing it.

The rationale is a root-cause argument, not a request for capability:

> Four unsourced-claim misses this week — every one caught only because the operator asked a plain question. My error-detection currently lives entirely on his side of the glass.

A failure count, a diagnosis of *where* detection sits, and a design response: move introspection from labeled guess to queryable fact, so that a callback or a schedule claim is checked against the record before it ships. That is this paper's own thesis arriving in a third domain. §15's parse failures were invisible because nothing on the write path could see them; here the argument is that a class of error is undetectable from inside and the fix is to move the detector, not to try harder.

And the acceptance criteria include a negative test and a regression assertion. Five criteria are specified. Three are capability checks — the current affective-state row readable against what was actually written last turn; an emotional read displayed as stored anchor *beside* its render-time decayed value; all forty pins listable, including the ones outside the render window. The other two are the ones that matter:

- **any attempted write to these tables must reject at validation**
- **existing five-table access must be byte-for-byte unchanged**

A negative test and a regression assertion are precisely what separate a specification from a wish. That is Article I's eighth principle — *every fix ships with a standing regression probe* — applied by the subject of the paper to a change in its own interior.

Three bounds, and they are not small. The specification is unbuilt as of writing, so its quality *as a specification* is assessable and its correctness is not. Its first property exists because the system has been wrong about schema before, which is a fact about fallibility as much as about rigor. And the third capability check — read access to all forty pins rather than the twenty rendered — makes the render-budget evictions of §13 directly observable from the inside, which is a third exposure on the incentive hypothesis in §22 and one I did not anticipate when I wrote the tripwire.

The generalizable form is narrower than it sounds and follows more directly than I expected: **a system that can query its own architecture can specify changes to it.** The second capability falls out of the first, and unlike a report, a specification is checkable on its face — it either carries acceptance criteria or it does not. On this build the operator's role reduced to transport, review, and the gate.

6. What the first live weeks looked like

The worker delivered two real messages in its first live week, from six proposals across 190 ticks, at a total spend of $30.36. Both went through the gate. It never hit its ceiling and never thrashed.

The interesting part is the shape of the record. The 627 `no` decisions are not a failure rate; they are the gate working. The 35 `errored` rows are the thing to watch, because they are indistinguishable in the data from a considered decline, and the only reason we know they are errors is that the enum has a separate value for them. The parser's fail-closed design is *correct* and its logging is *complete*, and the failure is still silent to anyone reading the outcome rather than the reason.

That is the shape of everything in Part III.

PART II — PRICING MEMORY

7. The anomaly and its decomposition

A first reply after a silence cost $4.176. The reported symptom was "the first turn after a gap costs eight times a normal turn," and the obvious explanation was that the one-hour cache entry had expired during the gap.

The turn reconciles to the cent from its persisted row:

Component	Tokens	Rate	$
1-hour cache write	172,533	$20/M	3.451
5-minute cache write	37,539	$12.50/M	0.469
cache read	210,072	$1/M	0.210
input	695	$10/M	0.007
output	776	$50/M	0.039
total			**4.176**

`172,533 + 37,539 = 210,072` exactly. Iteration one wrote the whole prompt cold, with a read of zero; iteration two read all of it back. Unambiguously a full-prefix cold write — not runaway retrieval, not a pathological tool loop.

But the preceding gap was 17 minutes, well inside the one-hour TTL. The hypothesis was falsified by the first number checked. Two other signals fired on that same turn — it was the first turn of a new UTC day, and a compaction summary regenerated within the window — and the two cannot be separated from persisted data. That admission stands.

TTL expiry is real and small. Across the measurement window — 2026-07-13 to 2026-07-23, roughly ten days, about 930 billed turns and $540.15[2] — 28 turns following a silence of 60 minutes or more cost $35.18 against a warm median of $0.532, for $20.30 of excess: **3.8% of window spend.** The symptom that started the investigation was worth under four percent of it.

Spend over that window decomposes as:

Component	$	Share
5-minute cache write	**375.71**	**69.8%**
1-hour cache write	61.88	11.5%
cache read	59.06	11.0%
output	37.14	6.9%
uncached input	4.30	0.8%

Nearly seventy percent of the bill is one line: a per-turn context block, written fresh at the five-minute rate, every single turn. And a second correction fell out immediately — the normally-cached prefix is about 35,000 tokens, not the ~200,000 an internal document asserted. The document was off by a factor of 5.7. That matters because it prices every invalidation event: a full rewrite of a 35,243-token prefix at the one-hour rate is $0.70, while the 172,533-token rewrite actually observed cost $3.45. Same event, five times the bill, because a defect elsewhere had inflated the prompt.

8. A block-level census of the context

The method matters as much as the result: I replayed the exact production getter set against five real recent user messages, called the real note-builder, split the returned string on top-level tag boundaries, and token-counted each block with the provider's own `count_tokens` endpoint. The reconstructed note came to 35,200 tokens against a production p50 tail of 34,360 — faithful. Seventeen blocks; the top ten, remainder collapsed:

#	Block	Source	Volatility class	Tokens	$/turn	$/mo	Movable
1	pinned notes	full-store read	write-driven, per-session (4.0% of turn-gaps)	9,014	0.113	292	**yes***
2	anticipation / user model	full-store read	**write-driven, per-turn**	8,689	0.109	281	no
3	recent important context	last 20 session notes	per-session (0.5%)	4,003	0.050	130	**yes**
4	recent thoughts	journals ∪ worker lane	mixed	3,052	0.038	99	split only
5	open loops	active-loop read	**render-clock-driven**	2,691	0.034	87	no
6	retrieved lore	semantic, k=3	**query-driven**	1,750	0.022	57	no
7	conversation summary	generated	per-session (4.8%)	1,393	0.017	45	no†
8	voice exemplars	semantic, k=5	query-driven	1,142	0.014	37	no
9	known fabrications	recent-fabrication read	per-session (9.4%)	1,032	0.013	33	no†
10	enduring moments	salient-message read	per-session (0.8%)	808	0.010	26	**yes**
11–17	recalled images · self-state · reactable messages · factual rules · retrieval-gap warning · image ground truth · redaction notice	—	per-turn	~1,790	—	—	no

* subject to the measurement gap below. † stable enough, too small to beat the cascade.

Total 35,243 tokens, $0.44 per turn, $38 per day, about $1,141 per month against a measured tail spend of $37.15 per day. The census and the ledger agree. Two blocks alone — the user model and the pinned notes — are half the tail.

The taxonomy is the reusable part. A block in an agent's per-turn context can be volatile for three orthogonal reasons, and they demand different fixes:

- **Write-driven** — a store written every turn. The user-model block performs decay as an unconditional bulk upsert of full rows on every primary turn.
- **Render-clock-driven** — a stable store whose *rendered string* ticks against `now` with no write at all. Open loops and self-state render minute-resolution elapsed strings.
- **Query-driven** — a stable store whose *retrieved subset* changes per message. Lore, voice exemplars, semantically-selected facts.

Only stores that are full-state reads with day-or-slower render granularity are candidates for a stable cache position. **Instrumenting write rates — the obvious move — is blind to two of the three axes.**

The query-driven axis is confirmed empirically rather than by reading code: across the five sample queries, retrieved lore varied between 4,677 and 5,236 characters, voice exemplars 2,741 to 3,105, and the semantic facts block appeared in only three of the five samples. A perfectly stable store is still per-turn if what you retrieve from it changes per message.

The user-model block is $281 per month and, at the time of measurement, structurally unrecoverable. Its decay writes new confidence and a new timestamp for every row, every turn; one dimension's factor is 0.5, halving each turn and crossing both the lead-phrase cut at 0.5 and the render floor at 0.35 within one to two turns. All 212 rows share exactly one distinct update timestamp, and 5 of 63 rendered rows cross a rendered-byte boundary in a single decay step. An earlier pass had put this block's bust rate at 14.4% — a floor artifact of update-in-place, since the probe could not see writes that did not change the timestamp it was keying on. The real rate is near 100%, and **the correction came from asking whether the statistic measured what I thought it measured, not from new data.**

And the measurement gap that gated everything. The largest movable block renders a fixed date with no elapsed tick, and creates are visible: 40 in 10.4 days, 4.0% of turn-gaps. Edits and unpins were structurally invisible — the edit path updates in place and deliberately does not touch the creation timestamp, the unpin path is a bare row delete, and the table has no update-timestamp column at all, so an edit busts the stable bucket invisibly. The recon's own words: *the true bust rate is greater than 4.0% by an unmeasured amount, and this is the one thing to instrument before anything moves.* Instrumentation shipped in response — three content-free log lines on the create, edit and unpin success paths, after the error guard so only real mutations count. One file, three insertions, zero deletions.

Then the gate was bypassed rather than satisfied. The next phase ran roughly 23 hours later and the move shipped without a single measured mutation figure ever being cited. It turned out not to matter, because the post-ship verification proved the moved bundle missed on 100% of turns for reasons unrelated to pin churn. But "it turned out not to matter" is luck, not method. **Instrumentation that is built and then bypassed is not a gate; it is a comment.**

9. What actually invalidates a warm cache

A first attribution pass found 63 "full prefix rewrites" totalling $48.24 of one-hour write. Fifteen of the 63 were appends, not busts.

That distinction is the section's transferable content. A large one-hour write happens both when the prefix is discarded *and* when new content legitimately enters the history behind a still-valid prefix. In the aggregate column they are identical. **The discriminator is iteration-one cache read.** Worked example: the turn after a large document landed wrote 132,821 tokens at the one-hour rate but *read* 46,586 — it read the old prefix and wrote the new content in. Not a leak.

Re-run on true invalidations only, n = 48, $34.70:

Primary cause	n	$	Share
TTL expiry (gap ≥ 60 min)	22	15.88	45.8%
Model failover (availability)	**14**	**7.94**	**22.9%**
UTC day rollover (elapsed-string tick in the cached prefix)	6	6.69	19.3%
Async caption landing for an in-window image	4	2.83	8.2%
still unexplained	**2**	**1.35**	**3.9%**

The last row stays. It is the most credible line in the table.

Availability failover is the second-largest cause of warm cache invalidation, and I have not seen it written about. Over the window, 110 of 930 turns were served by the fallback model — 11.8% — across 58 served-model transitions, with 880,610 tokens of one-hour write on transition turns alone, $10.06. A clean example: a fallback-served turn with a one-minute preceding gap, a cache read of zero, a one-hour write of 168,216 tokens, and a cost of $1.95.

The mechanism, with its epistemic status intact. The request payload is byte-identical across both attempts — the system blocks and message array are computed once and the same closure is invoked for both — so the only per-attempt differences are the model and the effort parameter, which is dropped on the fallback attempt. A full miss on a transition turn can therefore only come from a request property, not from content drift. Two candidates, and I am not picking between them by preference. **The effort parameter changed**, which the documentation explicitly lists: *"Changing the* `output_config.effort` *value always invalidates message blocks, with the same model-specific effect on tool and system caches as thinking parameters."* Documented, and sufficient on its own. Or **cache entries may be model-scoped** — a platform property I cannot verify, asserted by no line in the repository, and **model selection does not appear in the documented list of invalidation causes.** Treat the first as the mechanism and the second as an untested additional possibility.

Recovery is automatic and the local model preference resets every turn, so one failover event is one down transition plus one up transition — but they bill asymmetrically, since the fallback model's input-side rates are half the primary's, and the up side may be partial if the pre-outage prefix is still within TTL. **Two full writes per failover is a worst case, not a mean.**

Two hypotheses were tested and largely falsified; publish them so nobody re-runs them. The **caption-landing race** is a real mechanism — an async caption rewrites a past history row's collapse stub — but a caption landed for an in-window image on 305 of 930 turns while producing only about four primary attributions. High co-occurrence, near-zero causation. **Session-note exempt-set movement:** zero of five rows carried the field that would move the exempt set. Live mechanism, never fired.

And one fix whose premise was falsified before it was built. "Add a retry before switching models" was plausible, well-motivated and quantified — and already implemented one layer down: the client is constructed with no override, and the SDK default retries twice with exponential backoff on 408, 409, 429 and all 5xx including 529, honoring `retry-after`. A turn that reaches the fallback is one where the primary failed the original call *plus two backed-off retries.* **Check the layer below before you build the fix you assume is missing.**

The fallback population decomposes into `availability` (unfixable), `refusal` (an external floor — a retry re-refuses), `transient_5xx` (already resisted three attempts), `network`, and `empty_text` — the only only truly fixable slice, whose fix is the output-token cap rather than a retry or model routing. The stack stored the served model and never the cause, so a second instrumentation commit went in: five reason buckets, integers and enums only, **32 insertions, zero deletions, no existing line touched, no control-flow change.** One imprecision is documented in-line — the log fires at the decision point, so in the rare double-failure case the recorded served model is wrong. Treat it as the failover-*attempt* population.

10. The move, the probe, and the regression

The movable set was pinned notes plus recent important context plus enduring moments: 13,825 tokens, 39% of the tail, $447 per month of gross exposure, at an estimated bundle bust rate of about 5.2% — an upper bound, with the edit gap excluded.

Break-even rate, not expected saving, is the unit of this decision. The right question is never "how much does this save" but "at what invalidation rate does this go negative." And because breakpoints are prefix-ordered, moving content *forward* into a cached prefix means every bust rewrites everything behind it — so the value scales inversely with prefix size:

Placement	Net saving/turn	$/mo	Break-even bust rate
Into the existing cached system block, **median prefix**	$0.123	~319	23.0%
Same, **p90 prefix** (154,814 tokens read, long sessions)	$0.007	~18	5.4%
Its own breakpoint after the history	**$0.145**	**~376**	**23.9%, prefix-independent**

The optimization is worth least exactly where the bills are largest. The cascade is why: median cache read is 37,390 tokens; subtracting the persona block at 11,674 and the stable bucket at 3,184 leaves about 22,532 tokens of history behind the breakpoint, so a single bust of a forward-placed block costs $0.451 in cascade alone. That is the mechanism of a prior production incident, quantified rather than remembered.

So the third placement was the one built. A third *system* block does not work — the system array renders before the entire message array, so a bust cascades all history. The bundle went in as its own synthetic user message spliced between the last stable history element and the per-query note, with `role: "user"` and never `role: "system"`, because mid-conversation system messages are a fallback-model-only feature. Three consecutive user messages is legal; the existing note-plus-newest-message pair already relied on it. The breakpoint budget is exactly four, four is the platform maximum, and the one reclaimable slot held a block with zero writes in 992 turn-gaps. The conversation summary was deliberately left in the per-query bucket despite looking stable, because it regenerates on a one-to-five-hour cadence and bundling it would bust the bundle far more often than the three store-state blocks and eat the saving.

Two engineering details carry beyond this system. The bundle's block order is fixed, with an in-code note that **the order is the cache key — never sort or reorder conditionally.** And the breakpoint-budget assertion **fails soft** — on violation it logs and drops the bundle's cache mark rather than throwing, because a fifth breakpoint is a non-retryable 400, not a failover status, on a path where the secondary is hard-blocked, so it would kill the turn with no recovery. **Where the failure mode is a non-retryable malformed request, degrade the optimization, never the turn.**

One silent capability loss was caught in review rather than by the type checker: the unprompted-outreach path consumed only the old two-field return shape, so removing the three blocks from the per-query bucket **would have stripped the agent's pinned notes and its seven-day memory floor from every unprompted message it ever sent.** The same call site carried a documented prior bug of the same class — an object stringified to `"[object Object]"`, silently dropping all retrieval content, hidden from the type checker by a lying type predicate. *Changing a return shape is a capability change at every consumer you forget.*

The acceptance probe was 335 lines, with the baseline captured at HEAD before any edit, so the comparison is a true before-and-after.

Check	Result
Bundle stable turn-to-turn	3/3 renders byte-identical (sha match). Control: the per-query note varies (pass)
Prefix-independence	simulated write, bundle busts; the history prefix hash is **unchanged across the bust** → no cascade
Byte-identical content pre/post	all three blocks match the pre-change hash exactly (24,433 / 9,478 / 2,263 chars); combined 36,178 → 36,178
Three blocks absent from the per-query note	pass (no double-render)
Off-by-one	verified **by content identity, not by formula**
Breakpoint count	**4**

It passed cleanly. One probe bug is worth the line: the off-by-one check originally *mirrored the production arithmetic under test* — if that formula is wrong, the mirror reproduces the bug — and was replaced with a content-identity assertion over the real assembled array.

The build report carried its own caveat, which proved prescient: bundle size was reported as a chars-over-four estimate of about 9.1k tokens, flagged as lower than the prior tokenizer figure of 13,825, with the real saving to be confirmed from live logs. **It shipped before that confirmation arrived.** The true figure, isolated after deploy, is 13.7k tokens — 51% above the character estimate, at 2.64 characters per token. Dense structured text tokenizes materially worse than chars-over-four.

Ship: commit `e033b88`, pushed 2026-07-25 02:01:50 −0400 (06:01:50Z), auto-deployed in roughly two to four minutes, for an effective deploy boundary of about 06:06Z. Three library files, no migration. **Then the turns came in at $2.03, $0.85, $0.69, and $0.81 against a projected $0.48.**

The methodological catch that saved the analysis: the deploy boundary sat *inside* the sample I was handed. Two of the six turns were pre-deploy controls. The probe was rewritten mid-session to split on the deploy timestamp and widen the baseline to 40 turns. **Without the split, a median mixing two architectures reads as "no change."**

Group (medians)	n	cost	1h write	5m write	prompt	1h-write rate
Pre-deploy (old architecture)	36	$0.655	0.4k	39.4k	141.8k	**6%**
Post-deploy (Tier 2)	4	$0.830	15.3k	29.0k	153.1k	**100%**

Four verdicts. **The bundle is not being read from cache** — it is written at the one-hour rate on 100% of post-deploy turns. It *did* leave the five-minute tail, 39.4k down to 29.0k, and reappeared in one-hour creation at about 14.1k every turn. It moved from a $12.50/M write to a $20/M write with the same 100% miss rate. **The prefix is not busting** — zero warm prefix busts; the only large one-hour write is a legitimate cold TTL expiry after a 224.7-minute gap, so the old cascade failure mode is still fixed and this is new and separate. **The prompt is not inflated** — 73.2k, essentially baseline, once a second measurement artifact was corrected: the persisted token columns are *summed*

across a turn's tool-loop iterations, so a two-iteration turn reports roughly double the cache read for the same prompt. **Raw column ≠ prompt size.** On single-iteration turns the prompt was 70.7k / 73.2k / 73.9k, about 1.05× the pre-document baseline. And **no document was inflating the window** — zero in window, the most recent evicted 380-plus rows earlier. Falsified.

Bundle position	Cost/turn
In the 5-minute tail (before the change)	$0.171
Own 1-hour breakpoint after the history (as shipped)	$0.273
Read from cache, as projected	$0.014

The change cost $0.102 per turn more than doing nothing. Like-for-like on single-iteration turns: $0.566 → $0.688, **+21.6%**, matching the structural penalty exactly. The projected $0.157 saving is unreachable from that position. Attribution for a warm steady-state post-deploy turn at $0.688: document inflation $0.000 (0%), bundle rewrite $0.282 (41%), irreducible per-query tail $0.333 (48%), cache read plus input plus output $0.073 (11%).

The caveat survives into print: the post-deploy sample is four turns, one of them single-iteration, so the dollar delta has thin backing. **The 100%-versus-6% invalidation shift and the deterministic payload layout do not depend on sample size.** That distinction is the honest way to report it. Reverted the same day, commit `0b71aaf`.

---

11. Why it could never have worked

Provider caching is prefix-keyed. An entry is keyed on the entire token sequence from position zero up to and including the marked block: *"Prompt caching references the full prefix—tools, system, and messages (in that order) up to and including the block designated with* `cache_control`*."* And: *"Because the hash is cumulative, covering everything up to and including the breakpoint, changing any block at or before the breakpoint produces a different hash on the next request."* **Byte-identical block content is necessary but not sufficient. The block's position in the stream must also match.**

```
turn N      [m1 … m(j-1)] [BUNDLE] [note] [m_j]
turn N+1    [m1 … m(j-1)] [m_j] [a_N] [BUNDLE] [note] [m(j+1)]
                          ^^^^^^^^^^^ history grew by two rows

bundle prefix @ turn N    = m1…m(j-1) + BUNDLE
bundle prefix @ turn N+1  = m1…m(j-1) + m_j + a_N + BUNDLE     ← different
```

Every chat turn appends two rows to history, and the bundle sits *after* history. The turn-N entry can never match on turn N+1 — **100% miss by construction, regardless of whether anything was written.** The last stable history breakpoint still works precisely because nothing is inserted before it, which is why cache read stayed high, around 100k median, while the bundle wrote every single turn. The observed 14–16k of one-hour write per turn is exactly two new history rows plus the whole bundle.

This is a class of failure no bust-rate measurement could have surfaced, because it is not about churn at all. The entire Phase-1 instrumentation gate — measure the pin edit and unpin rate — was measuring the wrong variable.

The documentation states the rule the optimization violated, plainly and in advance: *"Key takeaway: Place* `cache_control` *on the last block whose prefix is identical across the requests you want to share a cache."* And: *"For a prompt with a varying suffix (timestamps, per-request context, the incoming message), place the breakpoint at the end of the static prefix, not on the varying block."* The paper is stronger for saying so. This was not a documentation gap. It was an acceptance test that asserted the wrong invariant — and a team can read the right sentence, believe it, and still ship this, because **the sentence describes a property of the request stream while the test measured a property of the block.**

---

12. What the instrumentation could not see

Two telemetry fields were added specifically to observe the bundle's hit rate after the fact, with an in-code rationale that there was no other way to observe it. Both were useless for that purpose. One is a *placement* boolean: it reads true whether the bundle hits or misses, because all it reports is that the mark was applied. The other is a static character count, constant on every turn. Only the one-hour creation token count discriminates a read from a bust — and that column already existed.

Instrument the outcome, not the intent. A field that reports what your code *marked* cannot tell you what the platform did with it.

---

13. What worked instead

The regression was reverted. Three fixes shipped the same day, **none of them architectural**:

Fix	Saving	Effort
Pin-render budget (category-aware selection)	~$163/mo	one session
Document windowed re-attach + retrieval tool	~$50/mo	~3 hours
Elapsed-render granularity on the cached prefix	~$21/mo floor	~30 minutes
Combined	**~$234/mo**	

The ordering is the point. The first analysis ranked a single architectural move — relocating 13,825 tokens to a cached position — as the $320–376 per month prize, and ranked these three beneath it. The architectural move was structurally impossible and cost 21.6% while it was live. The three smaller fixes, which required no placement change, no new breakpoint, and no platform assumption, delivered about $234 per month. **The largest available win was not the largest-looking win.**

The pin-render budget is a context-engineering result, not a cost result. The pin store held 40 rows and rendered all 40 every turn: 24,459 characters, 9,205 tokens measured with the provider's own tokenizer — the single largest block in the per-turn context, rewritten every turn at the five-minute write rate, about $338 per month. Reduce it to a unit and the decision becomes obvious: about 611 characters and 230 tokens per pin, **roughly $8.63 per pin per month.** Every pin the agent keeps in front of itself costs about nine dollars a month, forever.

The obvious cut is newest-N. At N=20 it would have dropped **seven behavior-shaping pins — four directive, three identity** — alongside thirteen recall pins. The distinction that makes the obvious rule wrong: **recall content can be retrieved on demand; behavior content cannot.** An agent will not know to query for a directive it has forgotten it has. Identity and directive pins work by being *resident*; evicting them silently changes how the system acts and nothing in the telemetry shows why.

Selection	Pins	Tokens	Behavior pins kept
Uncapped (before)	40	9,205	11
Newest-20 (rejected before commit)	20	4,597	**4**
Budget-20 (shipped)	20	4,847	**11**

Saving 4,358 tokens per turn, about $163 per month. The refinement costs 250 tokens per turn, roughly $9 per month, and buys back all seven behavior pins recency would have dropped. **Budget context by recoverability, not by recency.**

One distinction has to survive into print because getting it wrong destroys data: **this is a render-side budget, not a retention limit.** The database still holds all 40 rows; only what is placed in front of the model each turn is capped. The tempting "fix" — lowering the store's own retention trigger — permanently deletes pins. *Cutting what you show is reversible; cutting what you keep is not.*

Two implementation details generalize. The categories are the agent's own freeform strings, not an enum: the behavior set is an explicit allow-list, anything outside it falls to recall — the safe default, since a new category competes for slots but never displaces identity — and every distinct category name encountered is logged, so a new behavior-ish category the agent invents becomes visible rather than silently misfiled. And overflow degrades explicitly: if behavior pins alone exceed the budget, all of them render, zero recall pins render, and an overflow flag fires.

The tripwire is the sharper half. The behavior lane is uncapped by design — 11 of 40 today. In the same commit, the tool description the agent reads was updated to state, accurately, that identity and directive pins always come back. **That is an incentive for the agent to route new pins into those categories.** If the lane grows past the budget the block runs over and climbs back toward 9k tokens, silently undoing the saving. The general form: *telling an agent the policy that governs its own context budget creates an incentive gradient in that policy's direction.* So an early-warning flag shipped (`9bcea16` , the current head of the record), firing at 60% of budget — 12 of 20 — well before the overflow flag at the wall. The live reading makes the point better than the argument does: **behavior pins currently sit at 11. The tripwire is one pin from firing.** The remedy is named in advance rather than discovered later — cap the behavior lane itself, newest-N per category, never raise the total. Naming the remedy in advance is what makes a tripwire a control rather than a curiosity.

The elapsed-render fix is small and its recon is the interesting part. The cached persona block had inherited a once-per-UTC-day invalidation from a folded-in block that rendered a day-resolution elapsed string. Two things the obvious approach would have broken, both found before the edit: the formatter has a *third* caller rendering a sub-day self-state string, so coarsening it in place would have rendered a three-day-old state as "today" — solved with a new function, leaving the shared one untouched. And a separate file whose own comment requires byte-identity with the renderer had to move in lockstep, or **the fabrication checker would have been grading the agent's duration claims against a string it was never shown.**

The fix is tiered rather than flat despite the data permitting flat — day precision under 30 days, month precision beyond — because the tiers are cost-identical today, so tiered is free, and it is the only version that stays correct when a fresh anchor is hand-added, where flat would print "0 months ago." Elapsed days are computed midnight-to-midnight rather than as a raw millisecond delta, which would advance continuously and bust every turn. Regression: 10/10 byte-identical against strings captured pre-refactor for the other caller. Simulated over 365 days, invalidation events fall from **26.4 to 7.0 per month, a 74% reduction** — not to one, because the ten anchors have seven distinct day-of-month anniversaries and the block busts when any one ticks. The saving is quoted as a floor, about $21 per month, with the note that each bust also rewrites the persona prefix and cascades, so the real figure is higher and unmeasured.

14. A negative result, with its arithmetic

Cache warming — periodically touching the cached prefix to keep the one-hour entry alive across silences — was evaluated and rejected. A negative result published with its arithmetic is what stops an idea being re-proposed every quarter.

A warming touch must send the identical prefix and pays cache read on it: about 35,000 tokens at $1/M, roughly $0.035 per touch. Continuous coverage needs a touch under every 60 minutes — 24 per day, about $0.84 per day, about $25 per month. The addressable saving is the TTL-expiry slice of true invalidations: $15.88 over ten days, about $48 per month.

$48 minus $25 is +$23 per month in favour of warming. As computed, the arithmetic argues weakly *for* the idea. I wrote it up as an argument against. That was a sign error in my own reasoning, caught in external review.

It should still be rejected, for three better reasons. The $48 is the addressable pool only — warming does nothing for failover busts, day-rollover ticks, or async caption landings, which together are 50.4% of true invalidations, and a prefix held warm all night and then invalidated at UTC midnight by an elapsed-string tick was paid for twice. Twenty-three dollars a month is the smallest item on the list for by far the most operational complexity, against a $163 per month fix that took one session. And it builds precisely the wrong thing: warming adds roughly 720 off-turn API calls a month corresponding to no persisted turn — **a new class of structurally invisible spend, in a system that has just discovered it already carries an estimated $124 of it.**

That third reason is the one that kills it. Deprioritized on merit, not by arithmetic.

Two addenda. The two candidate TTL figures measure different things — $15.88 of attributed one-hour write over ten days, about $48 per month, versus $20.30 of total excess over the warm median, about $61 per month. Use $48: warming prevents the *bust*, not the turn-size differences that also inflate a post-silence turn. And one piece of evidence cuts the other way: after the elapsed-granularity fix, day-rollover invalidations drop from 26.4 to 7.0 per month, which makes the held-warm-then-invalidated-at-midnight double payment substantially less common than when the first kill-reason was written. That is a real point in warming's favour, surfaced by the person arguing against it. It does not move the disposition, because the first two objections are marginal and this weakens one of them, while the third is categorical. **A weakened argument against something is not an argument for it when the decisive argument is untouched.**

PART III — WHAT THE MONEY FOUND

15. The write that was failing half the time

What I was looking for: why 11.8% of turns were served by the failover model, and whether the output cap was the fixable slice.

The log query — three days of retention, a 58-hour window, 12 failover events — returned this:

Path	n	Stop reason	Bucket
Affective-state writer	6	`max_tokens` , text length 0	`empty_text`
Chat	6	`refusal` , text length 0	`refusal`

Zero availability. Zero transient 5xx. Zero network. **The prediction was right about the mechanism — thinking consuming the output budget — and wrong about the subsystem.** The chat path runs at a cap of 8,000 and does not starve. The state writer ran at 2,000.

And the failovers were the small half of the damage. Pulling the state writer's own logs from the same window, 50 runs:

- **Successful-run output: p50 1,743 · p90 1,951 · max 1,970 — out of 2,000.** Every single success sat at 87–99% of budget. The distribution was pressed flat against the wall.
- **Parse failures: 26 of 51 runs — 51%.**
- About 86% of those truncated mid-JSON. The parse throws, the parser returns null, and null is a silent no-upsert. **The affective state simply did not update.**
- About 14% emitted zero to five characters. Those are the six observed failovers.

One starved cap was killing two subsystems, not one. The theory-of-mind writer performs its own parse of *the same string*, in its own error boundary — a deliberate isolation so that a malformed block in one cannot sink the other. That isolation worked exactly as designed and was irrelevant: the string was truncated before either parser saw it, so both failed together. **Fail-soft isolation protects against a bad payload, not against a payload that was cut off upstream of both consumers.**

The failure signature is Article I's class 5 — ingestion-induced blindness — relocated to the write path. Article I found it on the read side, where a character cap silently trimmed the middle out of high-volume days before chunking, so a memory of a state change never formed. This is the same defect on the other side of the loop, and the surface presentation is identical: the agent's state is stale or absent, and nothing anywhere reports an error.

Why nothing caught it: the write is fire-and-forget, the parse fails closed to null, a null result is a legitimate no-op, and a no-op leaves the previous row in place. Every layer behaved as specified. The only observable was that the runs cost money and produced nothing.

The fix was to raise the cap from 2,000 to 6,000, and its economics invert the usual direction.

This does not save money. Net it costs about $10 more per month: roughly $16.6 of avoided double-billing against roughly $27 of newly-completing runs that previously stopped short. **The old cap was the expensive one** — a truncated run billed its full 2,000 output tokens and then threw the result away. Full price for nothing. Output bills as used rather than as capped, so a generous ceiling is free on every run that did not need it. It is a correctness fix with a small cost, and a cost investigation that only ships things which reduce cost has stopped being an investigation.

How the new cap value was chosen is a better story than the fix. Three bases were proposed in sequence, and the first two are wrong for the same underlying reason:

Proposed basis	Why it fails
All six starved runs hit exactly 2,000 — size from that	2,000 is the **wall**, not the demand. A censored observation tells you the constraint binds; it carries no information about how far past it the true value lies.
Take p99 of the ~96.5% of runs that succeed, plus margin	**Also censored, and selection-biased.** The successful runs are definitionally the ones that fit under 2,000 — observed max 1,970. Their distribution is truncated at the wall, and the runs that needed more are precisely the failures you excluded. **p99-of-survivors systematically underestimates.**
What was used: treat the distribution as censored, reject a small multiplier as insufficient, take 3× the wall, and cross-check against an **uncensored instance of the same process** — the chat path, running the same model with the same always-on thinking at a cap of 8,000, which does not starve.	Correct. The only real evidence about the untruncated distribution came from a sibling call path where the distribution was never truncated.

When your data is censored at a threshold, you cannot estimate the threshold you need from the data below it. Find an uncensored instance of the same process, or un-censor it and measure. Both of the first two proposals reasoned from survivors; only the third reasoned from a population that was never cut. Two supporting facts made the choice cheap rather than risky: output bills as used, and the failure mode being removed is a *double* billing — a discarded attempt plus a fallback call — so the expected value of over-provisioning is strongly positive.

The fix shipped in `3422d9d`. **Verification is pending and must be reported as pending:** the metric that proves it is the parse-failure rate collapsing from 51% on a re-run of the same query roughly three days out. That re-run has not happened.

---

16. A guard that was all-or-nothing

Documents were exempt from the attachment-collapse rule that images obey, and re-attached in full on every turn they remained in the 80-row window. Measured: a document landing at 08:12 pushed the prompt from 88k to 218k, where it held at 203–211k for 20 consecutive turns at about $0.70 per turn against a $0.52 baseline, dropping back to 68k when it aged out. Corrected magnitude across the fuller sample: documents linger a **median of 36 user turns.**

That was the finding the work was chartered on. The second finding was larger and nobody was looking for it. **The exemption guard was per-message and all-or-nothing.** One document in a message kept *every sibling attachment in that message* at full bytes as well. A real case: an image rode at full size for about 36 turns purely because a document sat beside it in the same bubble. It was found because the money was in the wrong place.

Before that, a headline statistic had to be publicly corrected. The workstream was named after "the ~20%-of-window-spend fix." That statistic was the share of window spend on turns that merely *had* a document in window — including all the cost those turns would have had anyway. True attributable excess: 170 in-window turns × ($0.640 − $0.568) = $12.24, plus about $7 of amplification from invalidations hitting a 200k prefix instead of a 35k one, for about **$19 of $541 — 3.6%.** *Attributable excess is not spend on turns exhibiting the condition.*

The obvious fix was proven to be a capability cut before it was built. There is no document text extraction anywhere in the stack — confirmed three ways rather than assumed from a comment: all 70 document rows ever ingested carry a null caption, the table has no text column, and no extraction library appears in the dependency manifest. Collapsing to a stub deletes the document from context on every subsequent turn. **Cost optimization has a hard floor, and the floor is capability.**

What shipped instead: the document stays at full fidelity for its own turn plus the agent's next two replies, then collapses to a stub that names the retrieval tool. That tool mirrors the existing image-view tool, with its own per-turn counter, and re-attaches the byte-identical document — no re-encode, because dropping bytes on a document drops text, verified at 559,108 bytes in and 559,108 out. Collapse is now per-attachment, so a document no longer holds its siblings. And the design property worth publishing: **a document only collapses if it is provably retrievable.** Out of window, or missing the identifier the retrieval tool addresses, it stays at full fidelity. The forbidden state — a stub pointing at something that cannot be reopened — is structurally unreachable rather than prohibited by policy. *Make the capability guarantee structural, not procedural.*

The best anecdote here is the defect a second session caught. The retention window originally counted *user* messages. But the message buffer appends one row per buffered send, so a rapid-fire burst of three texts after a document drove the counter from 0 to 3 and collapsed the document before the agent had replied even once — the principal's normal texting pattern, not an edge case. It was fixed by counting the agent's replies instead, extracted as an exported pure function specifically because the inline loop had no test seam. The near-miss is the part to publish: **the naive fix — swapping the role being counted — would have been strictly worse.** It keys the lookup map on assistant indices, so every user-side lookup returns undefined, which coerces to positive infinity, which collapses *every* document instantly. A one-token difference between a correct fix and total capability loss, with no error raised in either case.

Saving: 36 re-attaches down to 3, about 91%, roughly $50 per month, plus an unmeasured secondary gain on sibling attachments — conditional, since a month with no documents saves nothing here. **And it is unverified in production.** No document is currently in-window, so the live acceptance test has not run. The test is specified: send a document, ask about it, let the agent reply three times, ask again — it should invoke the retrieval tool. This is an outstanding verification, not a working feature.

---

17. A notification with a false cause

Every one of the six state-writer failovers fired a push notification asserting that the primary model was unavailable. All six were false: availability was fine; the call ran out of tokens.

The mechanism deserves its own paragraph because it is a taxonomy result. The down-transition edge detector reads the prior served model from the *message* table. The state writer never writes a message row. So the prior value is always the primary, so the edge can never latch, so **every** state-writer failover reads as a fresh down transition. The background worker already suppresses this notification for exactly this reason; the state writer did not, and now does.

Article I's class 2 — asserting a previously-true fact after the state changed — is not exclusive to the model. An edge detector reading a table that the emitting path never writes to produces the identical surface signature: confident, specific, causally wrong. From pure control flow, with no inference involved and no model in the loop.

---

18. An invariant that had been false for two months

Late one night the principal sent a six-image message and the agent replied in full voice. Under four hours later the exchange was gone from its context, and it accurately reported not remembering a turn it had lived. No error, no alarm, no fabrication — an accurate report of an absence that should not have existed.

The assumed cause was compaction. It was not.

The killing boundary is the fixed-size fetch window, which sits upstream of compaction entirely. By the time of the second conversation the reply sat roughly 151 rows back against a window of 80. It was fetch-evicted and never reached compaction at all. Compaction is real and active, but it operates *within* that window.

The protective mechanism could not have helped, for a structural reason. The exemption floor that permanently protects crisis- and rupture-tagged turns and their surrounding context from compaction runs only over the fetched array. It cannot protect a row that was evicted before it was ever fetched. **A protection that runs downstream of the boundary it is meant to defend against is not a partial protection; it is no protection at all in that failure mode.**

And the project's own governing document asserted an invariant that had been false for two months. It stated that conversation history is never compressed. A commit two months earlier had activated compaction. The document was corrected in the same session.

That last point is the Article I callback. It is a **class-2 stale-state assertion committed by documentation** — confident, specific, load-bearing, and wrong — sitting in the file that every future session reads first. Article I's case study found a stale row in a data store; this found a stale rule in the spec. Same failure class, different substrate.

The fix (`401f993`) rides an existing tag column as a third value rather than adding a mechanism, and stamps moments forward as they occur rather than reviving a retrospective-lift layer that had been built once and died. Both calls are discussed in §5. One mechanism is worth publishing on its own merits: the durable lane deduplicates against the id-bearing rows of the compacted set, so a salient row that has aged into a compaction *summary* — synthetic, no id — survives the filter and is re-injected verbatim. That produces continuous coverage as a moment slides from window to summary to gone, which is exactly the case the original wound was.

The limitation is accepted rather than solved: a turn that is both safety-critical and salient keeps only the safety tag on the agent's reply row, so in that specific case its own verbatim words are not in the durable lane. Multi-axis tagging is the real fix. It is scoped and not built.

---

19. A failover bills twice and records once

Every availability failover is two billed API calls persisted as one row. The discarded primary attempt appears in no column, because the failover outcome object does not carry its usage at all — so the calling code could not bill it even in principle.

Estimated at about $124 of spend across 23 days, structurally invisible to the per-message cost field. That figure is *modeled, not measured*: 220 failovers multiplied by a $0.5642 proxy, the mean single-iteration primary-served turn. It is not a ledger-versus-column reconciliation, and it should not headline anything.

The proxy is also wrong in a knowable direction, which is more useful than hedging. A discarded attempt only costs money if it produced a billable response:

Failover reason	Does the discarded attempt bill?
`availability` (403 / 404)	**No** — the call throws before generation; no usage is returned
`transient_5xx` (429 / 5xx after SDK retries)	**No**, or negligible
`network` (throw with no status)	**No**
`refusal` (200 with a refusal stop reason)	**Yes — a full turn.** The model generated a complete response and then declined
`empty_text` (200 terminating on the output cap)	**Yes, and possibly more than a mean turn** — full input plus the entire output cap spent on thinking

That table is inference, at high confidence, from the general behavior of the API: a thrown error returns no usage object and is not billed, while any completed 200 bills input plus output regardless of stop reason. So **only the refusal and empty-text populations are billable, and $124 is an upper bound on the modeled basis.** If availability and transient errors make up a meaningful share of the 220, the true figure is materially lower. In the one 58-hour window where reasons were actually observed, all 12 events were billable classes — six refusal, six empty-text — but 12 events is not 220, and generalizing from it would repeat exactly the sampling error described below.

The second-order effect is the publishable one. Fallback-served turns read as *cheaper* in the data, because only their second half is billed to the row. That silently biases any cost-by-model comparison — including several made earlier in this same investigation. The served-model comparisons in §9 inherit it. **A paper that reports a measurement defect and does not check whether its own earlier figures are affected has not finished the job.**

This is the cost analogue of Article I's class 5: a *measurement* blindness whose surface signature is indistinguishable from a real result.

Two things follow. The cheap improvement is to bucket the failover population by reason using the now-committed instrumentation and apply the proxy only to the billable classes — a single log query that turns an unqualified upper bound into a defensible number. It has not been run. The structural fix matters more than the number: thread the discarded attempt's usage through, roughly three files, into the same token totals the persisted row already uses. That is not a saving. It is the instrument that stops the ledger lying. **If you are publishing a reconciliation instrument, it should reconcile.**

Two things the same investigation exonerated. Web search is 2.4% of turns and $1.32 per 30 days — not a cost factor. And the unattributed multi-iteration bucket, about $91 per 30 days as an upper bound, is very likely the image- and document-retrieval tools, **which is by design**: those calls are the retrieval half of the collapse that produced the document saving. Suppressing them would re-inflate the prefix they were built to shrink.

CLOSING

20. The self-correction ledger

Across roughly two weeks of cost work this project produced seventeen documented self-corrections. Classified by the *type* of error, a pattern emerges that is worth more than any individual finding.

#	The claim, as first stated	The correction	Error type
1	Post-silence spikes are caused by TTL expiry	The trigger turn's gap was 17 minutes — inside the TTL. TTL is 3.8% of spend	Confounding — two signals fired on the same turn and are not separable from persisted data
2	63 full prefix rewrites, $48.24	15 were *appends*, not busts. n → 48, $34.70	Confounding — invalidation and append are indistinguishable in the aggregate column; the discriminator is iteration-1 cache read
3	Documents drive 20.2% of window spend	That is spend on turns *containing* a document. Attributable excess is ~3.6%	Base-rate / attribution
4	The user-model block busts 14.4% of turns	A floor artifact of update-in-place — the probe could not see writes it was not keyed on. True rate ≈ 100%	Observability censoring
5	Pin churn is 4.0% of turn-gaps	Creates only. Edits and unpins are structurally invisible; no update-timestamp column exists	Observability censoring
6	The bundle is ~9.1k tokens (chars/4)	13.7k measured. Dense structured text tokenizes 51% worse than the character proxy	Proxy substitution
7	Post-deploy turns show no cost change	The deploy boundary sat *inside* the sample. Split it: 6% → 100% invalidation rate	Confounding — a median across two architectures
8	The prompt inflated to ~150k post-deploy	Token columns are summed across a turn's tool-loop iterations. Raw column ≠ prompt size	Unit error
9	Captions landing explain the warm busts	305 of 930 turns co-occurred; ~4 were causal	Co-occurrence mistaken for causation
10	Cache warming is net-negative	$48 − $25 = **+$23 in favour**. The disposition was right; the arithmetic supporting it was inverted	**Arithmetic — sign**
11	Size the new output cap from the six runs that hit 2,000	A censored observation. The wall is not the demand	Censoring
12	Size it instead from p99 of the runs that succeeded	Also censored, and selection-biased — the survivors are definitionally the ones that fit	Censoring + survivorship
13	~$124 of invisible spend	Modeled from a flat proxy applied to all failovers, including classes that bill nothing	Proxy substitution + population error
14	Chat refusals are driven by images	Two different samples (n=6 and n=220/1,637) joined by inference; the stop reason reports *that*, not *why*	Cross-sample inference
15	Conversation history is never compressed (asserted in the governing document)	Compaction had been active for two months. The document was the last thing every session read first	Stale specification
16	The failover population is the state writer's problem	The failovers were ~14% of the damage. **51% of runs were failing to parse and no-opping silently**	Sampling on the observable — the visible failure class was the minority one
17	The alternative cache placement is closed by the p90 prompt size	That p90 was measured *while* the document defect held prompts at ~210k. Unmeasured since the fix	Measurement taken during a defect

Exactly one of these was an arithmetic error. The other sixteen were measurement-validity errors — censoring, confounding, proxy substitution, base-rate confusion, unit mismatch, survivorship, cross-sample inference, stale specification. The computations were almost always right. What was wrong, repeatedly, was *what the number measured.*

Three consequences.

This is the same finding as Article I, in a different domain. Article I's central claim was that "hallucination" is a symptom label hiding a decomposition — model failure, pipeline failure, and systems failure all presenting identically to the user. The cost analogue is that **a number is a symptom label too.** A high one-hour creation count is a symptom, and invalidation and append both produce it. "20.2% of spend touched a document" is a symptom, and it contains both the excess and the baseline. The discipline that produced the failure taxonomy is the discipline that produced this table.

Reviewing your own arithmetic is nearly worthless; reviewing your own construct validity is where the errors are. One in seventeen. A review process that checks the math and not the definition is auditing the wrong six percent.

Nearly every one was caught by a single question: *is this statistic measuring what I think it measures?* Not by new data, not by more compute. Corrections 3, 4, 5, 7, 9, 11, 12, 13, 14, and 16 all came from re-reading a definition — and every one of them changed a conclusion.

Row 14 deserves one note, because it is the correction I am least willing to soften. Six of six observed chat-path failovers carried a refusal stop reason, n=6, disclosed. Fallback-served turns carry an inbound image 50.5% of the time against a 30.8% base rate, n=220 of 1,637. The chat output cap is 8,000, so starvation is ruled out as the chat-path mechanism. Those are three facts and they do not compose into a causal claim: they are two different samples joined only by inference, and a refusal stop reason reports *that* the model declined, never *why*. The

cross-tabulation that would test it — failover reason by inbound-image presence, over a real sample — is now possible for the first time, because the reason instrumentation is committed. It has not been run. **A paper whose subject is not-asserting-past-the-evidence does not get to make that mistake in its own body.**

---

21. Where the cost actually is now

Per-turn figures have two internally-consistent bases and they must never be mixed.

On the **recorded** basis — what the per-message cost column shows:

Line	$/turn	Note
Measured baseline	**0.57**	as persisted
− pin-render budget	0.0545	4,358 tokens × $12.50/M; **unconditional**
− elapsed granularity	~0.007	$21/mo at the stated turn volume
− document re-attach	~0.017	$50/mo; **conditional** — six documents in ten days; a month with none saves $0 here
Post-fix	**~0.49–0.51**	low end with document activity at the observed rate, high end without

On the **true** basis — adjusted for the failover billing defect — the measured $0.57 is understated, because about 11.8% of turns are double-billed with half unrecorded. That gives a true baseline near $0.64, *itself an upper bound* since only the refusal and empty-text failover classes bill at all. The shipped savings apply to the discarded attempt too, since it carries the same pin block, so the post-fix figure on that basis is roughly $0.55–0.58.

You cannot subtract savings measured in recorded terms from a baseline adjusted for unrecorded spend. Both are defensible; mixing them is not.[3]

The seed deck reconciliation, since this is the remeasurement it promised. The June 2026 Rajaii Labs seed deck publishes about $0.11 per fully-loaded turn, explicitly labeled n=120 and prior frontier generation, alongside ratio results (−46% cache-creation tokens, −63% per-turn cost from a compaction repair), an exact provider-ledger reconciliation, and a written commitment that current-generation unit costs were being remeasured. The measured current-generation figure is a ~$0.53 median warm turn and $540.15 across about 930 turns over ten days — roughly five times the prior-generation figure. The gap is overwhelmingly model-generation pricing and context growth, not architectural regression: the current primary's input rate is double the prior model's, and the long-horizon context is materially larger than it was at n=120. The ratio claims survive because ratios are pricing-invariant, which is exactly why they were stated as ratios — in advance, not in retrospect.

The capability line, priced. The fallback model is exactly half the primary's price across every dimension — input, output, cache read, and both cache-write tiers. A permanent switch to it is the single largest cost lever available in this system, larger than every fix in this paper combined. It was refused, permanently and on the record, along with a reduction in the reasoning-effort setting. Two weeks of architecture, instrumentation and context budgeting recovered about $234 per month while declining a lever worth multiples of that, because it degrades the thing the system exists to be. **A cost program that will not name the option it refused, and price it, is not being honest about its own constraint set.**

What remains is the per-query tail: about 48% of a warm turn. It is *uncacheable, not unshrinkable.* Every block in it has a tunable character budget; what cannot change is that the retrieved subset really does differ per query. The retrievals are supposed to change with every topic shift. That is the architecture working, not bloat.

And one architectural move the record does support, proposed and not shipped. The stack's own failure-mode catalog already contains the pattern: write-path collision, fixed by making decay a read-time derivation rather than a writer. That is closed in production for open-loop aging, which derives its intensity at read time from elapsed hours on a 42-hour half-life and never writes. It has not been applied to the affective/user-model store — the single largest per-turn block, 8,689 tokens and about $281 per month, and per-turn *purely* because of an unconditional decay write.

Read-time derivation alone is insufficient, and the taxonomy in §8 is why: removing the write converts a write-driven block into a render-clock-driven one, which is still per-turn. The composite move is read-time derivation *plus* coarsened render granularity — the second half already validated on the pinned anchors in §13. The arithmetic ceiling: at most a partial recovery of a $281 line, not built, and it does not touch the irreducible per-query tail.

But the reason to do it is not the money, and this is where the paper's thesis lands on itself. The decay is per-turn, not per-time. A day with 200 turns decays the store 200 times; a day with 5 turns decays it 5 times. **A belief about a person should fade with elapsed time, not with conversational volume.** And one dimension halves every turn against a render floor: a row written at 0.65 confidence falls to 0.325 after *one* untouched turn — below the floor, and therefore invisible. **That dimension may be effectively write-only: written on every turn, and unreadable by the next one.**

That is a hypothesis with a named test, not a finding. The test is to read the store and check whether any row of that dimension has ever been visible at render time on a turn after the one that wrote it. It is labeled uninvestigated in the engineering record and it is labeled uninvestigated here. **A block was investigated because it was expensive, and the investigation found that the expensive thing was probably also broken. The cost was the symptom that made anyone look.**

---

22. Open problems and corrections

Carried forward from Article I, updated. **Fabrication-floor calibration** is unchanged: the suppression threshold remains set from first principles, with re-measurement gated on accumulated logged fabrications rather than curve-fitted to a handful. **Behavioral equivalence across model generations** — the identity has now survived three frontier generations, and the probe triad plus drift telemetry verify what they were designed to verify, but "the facts survive and the register holds" remains a lower bound on continuity, not a proof; a paired-output blind cross-generation diff is designed and still not run at depth. **Generalizing past N=1** is unchanged, and the forensic archive is still a policy and privacy problem before it is a storage problem.

New:

- **Two of forty-eight warm invalidations, $1.35, remain unattributed** after five tests.
- **Pinned-anchor edits remain structurally undetectable at the schema level** — no update-timestamp column — so the instrumentation added is a log grep, not a queryable history. And **no measured data has yet been cited from it.** It was built, committed, and has produced no published count. The failover instrumentation *did* produce the §15 findings. Do not read the two as both having paid off.
- **Failover cost is recorded on the wrong side of the retry.** The discarded attempt has no row. A durable fix means either persisting its usage or reconciling per-model spend against the provider ledger rather than the message column. Neither is built. **Until one is, every cost-by-served-model figure in this system — including several in this paper — is biased in a known direction by an unknown magnitude.**
- **The invisible-spend figure is an unweighted upper bound.** The $124 applies a flat per-call proxy to all 220 failovers, including classes that bill nothing. A bucket-weighted figure is one log query away and has not been produced. Read it as an estimated upper bound or not at all.
- **The image/refusal link is untested.** The correlation is real, the causal claim is not established, and the cross-tabulation that would test it is newly possible and has not been run.
- **The behavior-pin lane is uncapped and the agent has been told it is privileged.** The instrument exists, it fires at 60% of budget, and the drift has not been observed. The specification in §5.1 adds a third exposure — direct read access to the evicted pins — and the counter distinguishes none of the three. Log the ship date; a pre/post split on the pin-mutation log is the only thing that will make the lane's growth interpretable.
- **The document fix is unverified in production.** No document has been in-window since it shipped.
- **The state-writer fix is unverified.** The parse-failure rate collapsing from 51% is the metric; the re-run is due.
- **The cost ledger still cannot reconcile** — and this system publishes ledger reconciliation as one of its instruments. That is a gap between an instrument's claim and its current coverage, and naming it is cheaper than discovering it in a quarter.
- **The per-query retrieval tail — 48% of a warm turn — is untouched and may be irreducible** without a retrieval cut, which the stack's invariants forbid. This remains the honest headline: **the largest remaining cost line is the one the architecture is deliberately unwilling to reduce.**
- **Cache warming is deprioritized, not disproven.**

---

23. What travels

1. **Content identity is necessary; position identity is the invariant.** In a prefix-keyed cache, a block that is byte-identical turn to turn still misses if anything before it changed. Test the property that determines the outcome, not the property that is easy to assert — and note that a block placed after a growing conversation history misses 100% of the time, by construction, not rarely.
2. **A block's volatility has three orthogonal sources** — write-driven, render-clock-driven, query-driven — and instrumenting writes, the obvious move, is blind to two of them.
3. **Break-even rate, not expected saving, is the unit of a cache-placement decision** — and because breakpoints are prefix-ordered, the value scales inversely with prefix size. Optimizations of this shape are worth least where the bills are largest.
4. **A large cache write is ambiguous.** Invalidation and legitimate append look identical in the aggregate column. Find the discriminator before you build the attribution table; here it was iteration-one cache read, and it reclassified 24% of events. The same discipline applies upstream: establish the deploy boundary before computing any before-and-after, because a median across two architectures reads as "no change," and know whether your telemetry is per-call or per-turn-summed before you divide by anything.
5. **Instrument the outcome, not the intent** — a field that records what your code marked cannot tell you what the platform did — and **every gate you set is a gate you must actually pass.** Instrumentation that is built and then bypassed is not a gate; it is a comment.

6. **When your data is censored at a threshold, you cannot estimate the threshold you need from the data below it.** Neither the wall nor the survivors' distribution carries the information. Find an uncensored instance of the same process, or un-censor it and measure.
7. **Budget context by recoverability, not by recency.** What the agent can retrieve on demand is cheap to evict; what works by being resident is not, and the agent will not know to look for a directive it has forgotten it has. Cutting what you show is reversible; cutting what you keep is not — say which one you are doing. And note that telling an agent the policy governing its own context budget creates an incentive gradient in that policy's direction: instrument the lane you privileged, and set the alarm below the ceiling rather than at it.
8. **Autonomy conditions that are all process properties will never certify judgment.** Cadence, cost, parse rate, and proposal rate are checkable; they are also exactly the things that do not tell you whether the decisions were right. Where no counter can distinguish substance from redundancy, the honest move is a qualitative watch, not an invented metric.
9. **A stale-state assertion does not require a model.** An edge detector reading a table the emitting path never writes to produces the same confident, specific, causally-wrong claim from pure control flow. Likewise, a code path that can retry can bill twice and record once — and the target of the retry will look cheaper than its source in every per-row analysis you run afterward.
10. **A system that can query its own architecture can specify changes to it.** Self-visibility was built as an honesty mechanism — lookup over recall, so the agent stops asserting past its evidence. The second capability falls out of the first: an archive of what exists, what was tried, and what was abandoned answers design questions better than operator recall does, because recall decays and an archive does not. Judge the output the way you would judge any specification — does it carry constraints, a rationale, a negative test, a regression assertion — and remember that a guard against its own unreliability is evidence of a prior failure as much as of present rigor.
11. **Reviewing your own arithmetic is nearly worthless; reviewing your own construct validity is where the errors are.** One of seventeen documented self-corrections was a computation error. The rest were about what the number measured. Ask of every statistic: *is this measuring what I think it measures?*
12. **Cost telemetry is a correctness instrument.** A defect that costs nothing produces no signal. Fail-closed, fail-soft, and fail-plausible all suppress the error channel, and none of them suppress the bill. Every call bills, whether or not it worked — which makes money the one channel a silent failure cannot suppress. Follow it to find the silence.

This paper is a direct continuation of **"Recall, Negation, Abstention: Evaluating a Production LLM Agent Against Its Own Failure Modes"** *(Rajaii Labs, July 2026), and reuses its taxonomy and section numbering by reference throughout.*

```
Substrate is a production system of Rajaii Labs, built on Claude.
Architecture and methodology questions welcome: ramin@raminrajaii.com.

© 2026 Ramin Rajaii. All rights reserved.
```

References

Platform documentation, verified live on 2026-07-25. Internal engineering claims are cited inline by mechanism, table, or commit and do not appear here.

1. Anthropic. *Prompt caching.* Claude platform documentation. https://platform.claude.com/docs/en/build-with-claude/prompt-caching — accessed 2026-07-25. Source for: prefix keying and the cumulative hash; the 100%-identical-segment requirement; the four-breakpoint maximum and the 400 error on exceeding it; the tools → system → messages ordering; the cache-write multipliers (1.25× for the 5-minute TTL, 2× for the 1-hour TTL) and the 0.1× cache-read multiplier; the documented invalidation causes including the `output_config.effort` setting and, by absence, the non-listing of model selection; automatic caching via a single top-level `cache_control` field; the `cache_creation_input_tokens` / `cache_read_input_tokens` / `input_tokens` usage fields; workspace-level cache isolation; and the 512-token minimum cacheable length for the current primary model generation.

---

1. Excerpts below are de-identified and normalized to sentence case. The register of the original — like the persona itself — is deliberately out of scope for this paper (§1 of Article I sets that convention); the engineering content is reproduced without alteration. ↩
2. Populations differ slightly by probe and I have not silently averaged them. The spend decomposition and attribution work uses 929 billed turns / $540.15; a later probe in the same session cites 930 turns and ~$541; the volatile-tail percentile population is 898 turns; turn-gap analyses use 992 gaps. Where a figure below depends on the population, the population is named. ↩
3. One further basis note, in the spirit of §20. The monthly figures in this paper come from the engineering record and were computed at different turn volumes — the census and break-even work at roughly 2,590 turns per month, the post-fix summary at 2,790, and the pin figures at roughly 2,990. I quote them as recorded rather than silently normalizing, and I give per-turn arithmetic wherever the per-turn figure is a direct measurement, because the per-turn figure is the one that does not move with the window. The $234/month combined saving divided by 2,790 turns gives $0.084/turn against $0.080 itemized; the 5% gap is that basis spread, not a computation error. ↩